

Monitoraggio ambientale distribuito nel Compute Continuum

Aggregazione Edge-Cloud e valutazione della scalabilità

Alessandro Vassallo – Matricola 0366572

Università degli Studi di Roma Tor Vergata

Sistemi Distribuiti e Cloud Computing – A.A. 2025/26

Sommario—Questa relazione presenta un'applicazione distribuita sviluppata in Go per il monitoraggio ambientale nel compute continuum. Il sistema riproduce misure reali derivate da Sensor.Community e distribuisce l'elaborazione tra livelli Edge e Cloud mediante MQTT e Apache Kafka. La soluzione adotta un'aggregazione gerarchica basata su finestre temporali e un livello Cloud scalabile orizzontalmente tramite partizionamento Kafka e consumer group. La valutazione sperimentale analizza il comportamento del sistema al variare del numero di Cloud Worker e delle condizioni di comunicazione.

I. INTRODUZIONE

L'obiettivo del progetto è lo sviluppo di un'applicazione distribuita nel compute continuum per il monitoraggio di misure ambientali, distribuendo l'elaborazione tra livelli Edge e Cloud. Il caso di studio utilizza dati provenienti da sensori BME280 del dataset pubblico Sensor.Community, relativi a temperatura, umidità e pressione e raccolti nel mese di gennaio 2025.

Il dataset storico viene utilizzato per simulare un sistema di monitoraggio continuo. Le osservazioni vengono riprodotte preservandone le distanze temporali in event time mediante un replay accelerato e inviate ai rispettivi Edge Site. Ciascun Edge esegue una prima fase di validazione e aggregazione locale, mentre il livello Cloud combina successivamente i risultati provenienti dalle diverse zone per ottenere una vista complessiva del sistema. Questa suddivisione permette di distribuire la computazione lungo il continuum e di ridurre il numero di record trasferiti verso il Cloud, inoltrando aggregati locali anziché ogni singola osservazione.

Poiché il progetto è svolto individualmente, tra i requisiti alternativi previsti dalla traccia è stata scelta la scalabilità orizzontale. La valutazione sperimentale studia quindi il comportamento del sistema al variare del numero di Cloud Worker, mantenendo invariato il workload, e analizza inoltre l'impatto delle condizioni di rete sulla pipeline distribuita.

I componenti runtime dell'applicazione sono sviluppati in Go, containerizzati tramite Docker e distribuiti su AWS mediante un processo di deployment automatizzato. Il preprocessing offline del dataset è invece realizzato in Python.

II. DATASET E COSTRUZIONE DEL WORKLOAD

Lo scenario è stato costruito a partire dal dataset relativo ai sensori ambientali BME280. È stata considerata la finestra temporale di gennaio 2025 e il dataset, di dimensione pari a

circa 6.36 GB, è stato analizzato in modo incrementale per evitare il caricamento completo in memoria.

Una prima fase di preprocessing ha permesso di verificare la qualità dei dati, filtrare le osservazioni esterne alla finestra temporale e individuare i sensori dotati di coordinate geografiche valide. Delle variabili disponibili, le coordinate geografiche sono state utilizzate per la costruzione della topologia, mentre temperatura, umidità e pressione sono state mantenute come misure da riprodurre durante l'esecuzione. Le variabili relative all'altitudine e alla pressione al livello del mare, completamente prive di valori, sono state escluse.

Lo studio è stato limitato all'area europea. Per ogni sensore è stata determinata una posizione rappresentativa e le coordinate geografiche WGS84 sono state proiettate nel sistema metrico EPSG:3035, in modo da applicare il clustering su distanze spaziali coerenti. La popolazione risultante comprende 5102 sensori europei localizzabili.

La suddivisione in Edge Site è stata ottenuta tramite K-Means sulle coordinate proiettate. Sono stati valutati valori di k compresi tra 5 e 15 utilizzando il Silhouette Score come criterio principale di selezione, affiancato dall'inertia e dal raggio massimo dei cluster come indicatori diagnostici. La configurazione selezionata è risultata $k = 13$, con Silhouette Score pari a circa 0.521; l'elbow method individua invece $k = 9$, utilizzato solamente come riferimento diagnostico.

Una volta determinata la topologia sull'intera popolazione europea, è stato costruito il workload sperimentale selezionando 150 sensori. I sensori sono stati distribuiti nel modo più uniforme possibile tra i 13 Edge Site (11–12 sensori per Edge). La selezione all'interno di ciascun Edge è stata effettuata utilizzando un seed pseudo-casuale fissato. In questo modo ciascun Edge partecipa agli esperimenti con un numero comparabile di sensori, pur mantenendo le differenze naturali nel tasso di generazione degli eventi.

Infine, la fase di preprocessing produce due tipi di artefatti utilizzati durante l'esecuzione: il file di topologia, che associa i sensori selezionati agli Edge Site, e 13 shard di replay, uno per ciascun Edge, contenenti identificativo del sensore, timestamp, pressione, temperatura e umidità.

III. ARCHITETTURA DEL SISTEMA

L'architettura realizzata è una pipeline distribuita nel compute continuum, organizzata tra livelli Edge e Cloud come

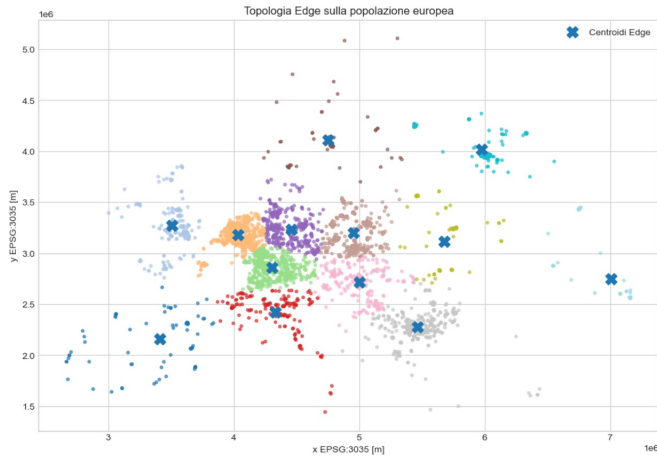


Figura 1. Topologia geografica della popolazione europea e centroidi dei 13 Edge Site.

mostrato in Fig. 2. Il sistema riproduce le osservazioni dei sensori selezionati dal dataset storico, le elabora localmente negli Edge Site e propaga verso il Cloud risultati progressivamente aggregati.

La pipeline è composta dai seguenti stadi logici.

- 1) **Simulator:** Per ciascuno dei 13 Edge Site è presente un Simulator che riproduce gli eventi dei sensori assegnati preservandone le distanze temporali in event time, opportunamente accelerate. La telemetria viene pubblicata tramite MQTT verso il broker Mosquitto associato al sito.
- 2) **Elaborazione Edge:** Ogni Edge Site comprende un broker MQTT e un Edge Gateway. Il gateway riceve e valida le misure di temperatura, umidità e pressione e le aggrega localmente su finestre temporali di durata configurabile. Questa elaborazione riduce il numero di record inoltrati verso il Cloud, trasferendo statistiche aggregate anziché le singole osservazioni.
- 3) **Trasporto Edge-Cloud:** Gli aggregati prodotti dagli Edge vengono pubblicati sul topic Kafka `edge-aggregates`. Kafka disaccoppia produttori e consumatori e mantiene l'ordinamento dei record all'interno di ciascuna partizione [2]. Gli Edge sono assegnati staticamente alle partizioni prima dell'esecuzione. L'assegnamento viene calcolato considerando il numero di eventi dei replay shard, con l'obiettivo di ottenere una distribuzione del carico più uniforme.
- 4) **Elaborazione Cloud:** I Cloud Worker appartengono allo stesso consumer group Kafka. Durante una run ogni partizione viene assegnata a un solo Worker del gruppo, mentre un Worker può elaborare una o più partizioni. Lo stato di aggregazione è mantenuto separatamente per ciascuna partizione. I contributi degli Edge vengono combinati su finestre temporali di durata configurabile, definita come multiplo

della finestra Edge. Aumentando il numero di Worker, le partizioni possono essere distribuite tra un numero maggiore di esecutori e quindi elaborate in parallelo, fino al limite imposto dal numero di partizioni del topic. Il numero di Worker viene fissato prima dell'avvio del replay e variato tra esecuzioni differenti.

5) **Aggregazione globale e persistenza:**

I risultati dei Cloud Worker vengono pubblicati sul topic `cloud-partition-aggregates` e consumati dal Global Aggregator. Quest'ultimo combina i contributi delle diverse partizioni relativi allo stesso intervallo temporale e produce il risultato globale della finestra. Nel deployment AWS tali risultati vengono persistiti in PostgreSQL tramite Amazon RDS.

Il numero di partizioni Kafka e il numero di Cloud Worker sono configurabili separatamente. Le partizioni determinano la suddivisione logica del workload e pongono il limite superiore al parallelismo del consumer group, mentre il numero di Worker determina quanta parte di tale parallelismo viene effettivamente sfruttata.

Nella campagna sperimentale il numero di partizioni e l'assegnamento degli Edge rimangono invariati, mentre viene modificato soltanto il numero di Cloud Worker tra esecuzioni differenti. Le stesse partizioni vengono quindi ridistribuite tra un numero diverso di esecutori senza modificare il workload o i risultati attesi. Questa separazione permette di valutare la scalabilità orizzontale del livello Cloud mantenendo costante la suddivisione logica dei dati.

IV. IMPLEMENTAZIONE

A. Aggregazione Edge e Cloud

L'elaborazione utilizza finestre tumbling basate sull'event time. Ciascun Edge mantiene una finestra corrente e assegna ogni evento alla finestra determinata dal relativo timestamp. L'Edge mantiene una sola finestra corrente: quando il timestamp di un evento determina il passaggio a una finestra successiva, quella precedente viene finalizzata. Eventuali eventi appartenenti a finestre già chiuse vengono considerati fuori ordine e scartati.

Per ogni grandezza ambientale vengono mantenuti il numero di valori validi e non validi, la somma, il minimo e il massimo. Al momento della chiusura della finestra viene inoltre calcolata la media sui soli valori validi.

Le statistiche sono state scelte in modo da poter essere combinate nei livelli successivi senza conservare le singole osservazioni. Per ottenere un aggregato a partire da più contributi, vengono sommati il numero di valori validi e le rispettive somme, mentre il minimo e il massimo sono ricavati considerando i valori estremi dei contributi che contengono almeno un'osservazione valida. La media viene quindi calcolata a partire dalla somma complessiva e dal numero totale di valori validi.

In questo modo la media globale viene ottenuta pesando implicitamente ciascun contributo in base al numero di osservazioni che contiene, evitando di calcolare una semplice media

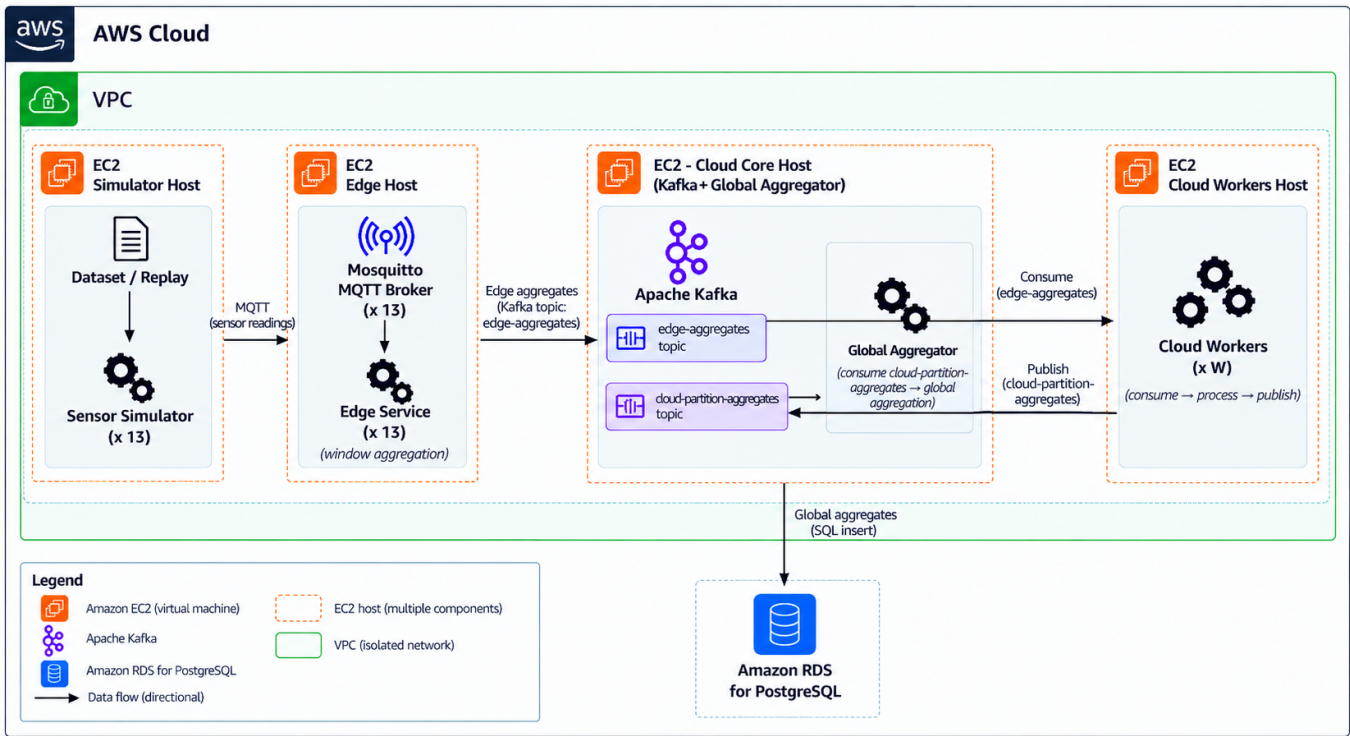


Figura 2. Deployment distribuito su AWS e flusso dei dati tra Simulator, Edge, Kafka, Cloud Worker, Global Aggregator e PostgreSQL. Amazon RDS appartiene alla stessa VPC ed è rappresentato separatamente nella parte inferiore esclusivamente per leggibilità.

delle medie locali, che sarebbe errata quando gli aggregati hanno numerosità differenti.

Le finestre Cloud hanno durata configurabile, ma devono essere un multiplo intero della durata delle finestre Edge. Ogni *EdgeAggregate* appartiene quindi interamente a una sola finestra Cloud. I Cloud Worker combinano gli aggregati ricevuti per partizione Kafka e producono un *CloudPartitionAggregate*; il Global Aggregator applica infine la stessa operazione di riduzione ai contributi delle diverse partizioni relativi allo stesso intervallo temporale.

B. Modello di concorrenza

L'implementazione in Go separa le operazioni di I/O dalla logica di aggiornamento dello stato delle finestre. Le goroutine vengono utilizzate principalmente per disaccoppiare le comunicazioni di rete dall'elaborazione, mentre gli aggregatori mantengono un modello di aggiornamento sequenziale.

Nel Simulator il replay temporale e la pubblicazione MQTT sono separati mediante una coda bufferizzata. Il ciclo principale produce gli eventi secondo la timeline accelerata, mentre una goroutine dedicata ne gestisce l'invio verso il broker MQTT. L'inserimento nella coda è non bloccante: quando la capacità configurata viene esaurita, l'evento viene registrato come drop locale invece di ritardare il replay.

Nell'Edge, il processing degli eventi e la pubblicazione verso Kafka sono eseguiti in due goroutine distinte collegate

tramite un channel. Il primo stadio aggiorna sequenzialmente lo stato della finestra Edge, mentre il secondo effettua la serializzazione Avro, il batching e la pubblicazione sincrona degli aggregati verso Kafka.

Nei Cloud Worker, per ogni partizione Kafka assegnata viene avviata una goroutine dedicata alla lettura. I record acquisiti confluiscono in un unico channel e vengono successivamente elaborati in modo sequenziale dal ciclo principale del Worker. Il parallelismo del livello Cloud deriva quindi principalmente dall'esecuzione di più Cloud Worker e dalla distribuzione delle partizioni Kafka tra essi, non da aggiornamenti concorrenti dello stato all'interno dello stesso Worker.

Il Global Aggregator non introduce concorrenza esplicita nel ciclo applicativo di consumo: i record vengono acquisiti da Kafka, elaborati e committati in sequenza. Questa scelta mantiene serializzati gli aggiornamenti dello stato globale e semplifica la gestione delle finestre e della frontiera event-time.

C. Progresso event-time e completamento

La chiusura delle finestre Cloud è governata dal progresso in event time delle sorgenti e non da timeout in processing time. Ogni *EdgeAggregate* contiene il campo *CompleteThrough*, che rappresenta il progresso in event time del relativo Edge e svolge un ruolo analogo a un watermark per sorgente: certifica che l'Edge non produrrà più

aggregati con estremo superiore precedente o uguale a tale valore.

Quando l'arrivo di un evento determina il passaggio a una nuova finestra, l'Edge emette l'aggregato della finestra precedente e può avanzare `CompleteThrough` fino all'inizio della nuova finestra. In questo modo vengono certificate anche eventuali finestre intermedie prive di eventi, per le quali gli Edge non producono aggregati espliciti.

Il Cloud mantiene separatamente il progresso di ciascun Edge appartenente alla partizione Kafka elaborata e combina tali informazioni per determinare la frontiera di avanzamento della partizione, utilizzata per stabilire quando una finestra Cloud può essere considerata completa. Indicando con C_e il valore `CompleteThrough` dell'Edge attivo e e con S il massimo disallineamento ammesso tra le sorgenti, la frontiera è determinata da

$$F_P = \max \left(\min_{e \in E_P} C_e, \max_{e \in E_P} C_e - S \right). \quad (1)$$

Finché almeno un Edge attivo non ha ancora comunicato il proprio progresso, la frontiera non può avanzare. In condizioni normali essa coincide con il minimo progresso degli Edge della partizione; il secondo termine impedisce invece che una sorgente eccessivamente in ritardo blocchi indefinitamente la finalizzazione delle finestre.

Una finestra Cloud viene emessa quando il proprio estremo superiore non è successivo a F_P . Nella campagna di scalabilità e nelle configurazioni conservative S viene mantenuto sufficientemente ampio da non forzare l'avanzamento della frontiera; la finalizzazione dipende quindi dal progresso naturale degli Edge della partizione. Nelle configurazioni bounded viene invece utilizzato un valore inferiore, che consente alla frontiera di avanzare rispetto a una sorgente eccessivamente in ritardo.

Un aggregato relativo a una finestra già superata dalla frontiera è considerato *late*: non modifica né riapre il risultato già emesso. Il progresso contenuto nel record può comunque essere utilizzato per aggiornare lo stato dell'Edge e consentirgli di riallinearsi alle altre sorgenti.

Poiché il workload è ottenuto dal replay di un dataset finito, è inoltre necessario determinare esplicitamente la conclusione del flusso. Al termine del proprio replay ciascun Simulator invia un marker di fine input al rispettivo Edge. L'Edge pubblica l'ultima finestra ancora aperta e, successivamente, un `EdgeEndOfInput`. Quando tutti gli Edge appartenenti a una partizione sono terminati, il Cloud Worker emette gli eventuali aggregati ancora pendenti e infine un `PartitionEndOfReplay`.

Il Global Aggregator utilizza il campo `CompleteThrough` contenuto nei `CloudPartitionAggregate`, insieme ai marker terminali delle partizioni, per distinguere un contributo non ancora arrivato da un contributo effettivamente assente. Il progresso globale è calcolato con la stessa politica bounded applicata dal Cloud agli Edge. Indicando con C_p il progresso della partizione sorgente attiva p e con S_G il massimo disallineamento ammesso, la frontiera globale è

$$F_G = \max \left(\min_{p \in P_A} C_p, \max_{p \in P_A} C_p - S_G \right), \quad (2)$$

dove P_A è l'insieme delle partizioni non ancora terminate. Finché una partizione attiva non ha comunicato alcun progresso, la frontiera globale non avanza. Una finestra globale può essere prodotta quando, per ogni partizione sorgente, è disponibile il relativo aggregato oppure il progresso, la frontiera globale o il marker terminale certificano che non ne verranno prodotti altri per quell'intervallo. Un contributo che raggiunge il Global dopo che la relativa finestra è stata superata da F_G viene registrato come *late* e non riapre il risultato già emesso. Il replay termina quando tutte le partizioni hanno raggiunto lo stato terminale e non rimangono finestre pendenti.

D. Comunicazione tra i componenti

1) *MQTT tra Simulator ed Edge*: La comunicazione tra Simulator ed Edge utilizza MQTT. Gli eventi dei sensori sono serializzati in JSON e pubblicati con QoS 0. In questo tratto della pipeline vengono trasmesse le singole osservazioni dei sensori; JSON è stato scelto per la semplicità della rappresentazione e per la facilità con cui i messaggi possono essere ispezionati durante lo sviluppo e il debugging.

La telemetria utilizza QoS 0, evitando il costo di una conferma applicativa per ogni singola osservazione. Il marker di fine replay utilizza invece QoS 1 e viene considerato completato dopo la ricezione del relativo PUBACK, poiché è utilizzato per segnalare esplicitamente la terminazione della sorgente.

2) *Kafka tra Edge e Cloud*: La comunicazione tra Edge e Cloud e tra Cloud Worker e Global Aggregator è affidata ad Apache Kafka. In questo tratto i messaggi sono serializzati in formato Apache Avro. Avro utilizza una rappresentazione binaria e permette di definire esplicitamente struttura e tipi dei record scambiati, caratteristica utile per gli aggregati composti da campi numerici e temporali. Gli schemi Avro sono inclusi direttamente nell'applicazione e condivisi dai producer e dai consumer.

Gli Edge utilizzano writer Kafka sincroni e attendono l'acknowledgment delle scritture prima di proseguire. Gli aggregati appartenenti allo stesso Edge e il relativo `EdgeEndOfInput` sono scritti sulla medesima partizione; prima di pubblicare il marker terminale vengono completate le scritture degli aggregati precedenti.

Analogamente, il Cloud pubblica per ciascuna partizione gli eventuali `CloudPartitionAggregate`, che includono nel campo `CompleteThrough` l'informazione di progresso della partizione, e successivamente, al termine del replay, il relativo `PartitionEndOfReplay`. Tale ordinamento garantisce che il consumer downstream riceva i dati prima delle informazioni che ne certificano il completamento.

I Cloud Worker effettuano il commit degli offset Kafka dopo l'elaborazione dei record e la pubblicazione degli eventuali output. I commit possono essere raggruppati in batch configurabili per ridurne la frequenza; la ricezione di un marker terminale forza invece il commit degli offset ancora pendenti.

Il Global Aggregator effettua il commit del record dopo l'elaborazione e, quando configurato il sink PostgreSQL, dopo la scrittura del risultato. Gli aggregati utilizzano identificativi deterministici, che consentono anche di riconoscere eventuali duplicati.

E. Deployment e piattaforma software

L'applicazione è sviluppata in Go e ciascun componente principale viene distribuito come container Docker. La configurazione del sistema è descritta tramite file YAML, dai quali il programma `deploygen` genera automaticamente i file Docker Compose necessari all'esecuzione. In questo modo parametri quali numero di Cloud Worker, numero di partizioni Kafka, dimensione delle finestre e parametri di batching possono essere modificati senza intervenire sul codice applicativo.

Per il deployment distribuito l'infrastruttura AWS è definita mediante Terraform. Sono previsti quattro host EC2 con ruoli distinti: Simulator, Edge, Cloud Core e Workers. L'host Simulator esegue i processi di replay, l'host Edge ospita i broker Mosquitto e gli Edge Gateway, l'host Workers esegue le repliche dei Cloud Worker, mentre Cloud Core ospita Kafka e il Global Aggregator. La separazione consente di far attraversare realmente la rete ai flussi Simulator-Edge, Edge-Cloud e Cloud Worker-Global Aggregator e di modificare indipendentemente il numero di worker utilizzato negli esperimenti.

Nel deployment AWS i risultati globali finalizzati vengono persistiti in un'istanza PostgreSQL gestita tramite Amazon RDS. Il database non è esposto pubblicamente ed è raggiungibile dal solo Cloud Core. L'infrastruttura, la configurazione applicativa e l'avvio degli esperimenti sono gestiti tramite script dedicati, in modo da rendere ripetibile la preparazione dell'ambiente e la raccolta degli artefatti prodotti da ogni esecuzione. Prima di ogni run che utilizza questo sink, il runner verifica PostgreSQL ed esegue esplicitamente il `TRUNCATE` della tabella `global_aggregates`, così che essa contenga soltanto le finestre finali dell'esecuzione corrente.

Le principali librerie utilizzate dall'applicazione sono Eclipse Paho per MQTT, `kafka-go` per la comunicazione con Kafka, `goavro` per la serializzazione Avro e `pgx` per l'accesso a PostgreSQL. Il preprocessing offline del dataset è invece realizzato in Python mediante `pandas`, `scikit-learn` e `pyproj`.

V. VALUTAZIONE SPERIMENTALE

A. Ambiente e metodologia sperimentale

La valutazione risponde a due domande. La prima riguarda la capacità del livello Cloud di elaborare lo stesso workload riducendo il ritardo di produzione dei risultati all'aumentare delle repliche. La seconda riguarda il compromesso tra latenza e completezza quando una sorgente logica viene rallentata sul collegamento Edge-Cloud. Le prove sono eseguite sul deployment AWS di Fig. 2.

Il workload definitivo comprende 2 106 784 osservazioni distribuite tra 13 replay shard. Il replay usa un fattore di accelerazione pari a 20 000; tale valore è stato scelto mediante

una calibrazione preliminare in modo da esercitare la pipeline mantenendo complete le esecuzioni utilizzate nella campagna di scalabilità. La configurazione comune alle campagne è riassunta in Tab. I.

Tabella I
CONFIGURAZIONE COMUNE DELLE CAMPAGNE SPERIMENTALI.

Parametro	Valore
Edge Site / Simulator	13
Partizioni <code>edge-aggregates</code>	6
Finestra Edge	5 min
Finestra Cloud	15 min
Fattore di accelerazione	20 000
Batch producer Edge	10
Batch commit Cloud	100

Per ogni run vengono conservati configurazione effettiva, log, metriche dei container, serie temporali del consumer lag e aggregati finali PostgreSQL. La riconciliazione verifica gli eventi offerti dai Simulator, quelli accettati e processati dagli Edge, gli eventi presenti nei risultati globali e quelli eventualmente scartati come late. Vengono inoltre controllati errori di pubblicazione, duplicati, finestre incomplete, terminazione del protocollo e lag finale.

Nella campagna di scalabilità una run è ammessa al confronto prestazionale soltanto se il risultato globale include il 100% degli eventi attesi, senza contributi late o perdite a monte.

Negli esperimenti bounded, invece, i contributi late costituiscono un effetto previsto della politica studiata e non rendono da soli non valida la run: devono essere contabilizzati e spiegare interamente la differenza tra gli eventi elaborati e quelli inclusi nel risultato globale. Eventuali drop nei Simulator, negli Edge o nell'emulazione di rete rimangono invece condizioni di invalidità della run.

La metrica principale utilizzata in entrambe le campagne è la *global emission latency*, cioè il ritardo con cui una finestra globale viene emessa rispetto all'istante nominale associato alla sua fine nella timeline accelerata del replay.

Indicando con t_0 l'istante di avvio del replay, con e_0 l'origine temporale del dataset, con f il fattore di accelerazione e con w_e la fine della finestra in event time, l'istante nominale associato alla finestra è

$$d(w_e) = t_0 + \frac{w_e - e_0}{f}. \quad (3)$$

Tale istante rappresenta il momento in cui la fine della finestra verrebbe raggiunta da un replay ideale eseguito con il fattore di accelerazione configurato e viene utilizzato come riferimento temporale per la misura della latenza.

La *global emission latency* è quindi calcolata come differenza tra l'istante effettivo di emissione della finestra da parte del Global Aggregator e $d(w_e)$. L'istante di emissione viene registrato prima del completamento della scrittura su PostgreSQL; la metrica rappresenta pertanto il ritardo di completamento dell'elaborazione della finestra e non il tempo necessario alla persistenza nel database.

Per descrivere la distribuzione delle latenze vengono riportati la mediana (p50), il 95-esimo percentile (p95) e il 99-esimo percentile (p99). La mediana rappresenta il comportamento tipico delle finestre, mentre p95 e p99 caratterizzano la coda della distribuzione, indicando il valore entro cui ricade rispettivamente il 95% e il 99% delle emissioni. Viene inoltre riportata la durata complessiva della run, mentre consumer lag e utilizzo delle risorse sono raccolti come metriche di controllo.

Poiché il ritmo di ingresso è determinato dal replay, un throughput medio calcolato sul numero di eventi elaborati rifletterebbe principalmente il rate di ingresso imposto e non la capacità massima dei Worker. La valutazione della scalabilità si concentra pertanto sulla latenza di emissione, verificando al tempo stesso la completezza dei risultati e l'assenza di backlog residuo.

Il lag Kafka è calcolato rispetto agli offset committati. Poiché i Cloud Worker effettuano il commit in batch, il lag può includere temporaneamente record già elaborati ma non ancora confermati tramite commit. Per questo motivo viene analizzato insieme alla relativa serie temporale e al valore finale, senza interpretarne automaticamente il massimo come backlog applicativo effettivo.

B. Scalabilità del livello Cloud

La campagna usa un workload fisso e varia il numero di Cloud Worker tra $W \in \{1, 2, 3, 4, 6\}$. Il topic mantiene sei partizioni in tutte le prove: workload, partizionamento e configurazione per Worker rimangono invariati, mentre varia il numero di esecutori tra cui Kafka distribuisce le partizioni. Sei Worker costituiscono il limite di parallelismo utile della configurazione; repliche ulteriori rimarrebbero senza partizioni assegnate.

Ogni run utile al calcolo ha elaborato 2 106 784 eventi, prodotto 2710 finestre globali complete, senza late record, duplicati o errori di protocollo, e terminato con lag nullo. La Tab. II riporta la media delle repliche.

Tabella II
RISULTATI DELLA CAMPAGNA DI SCALABILITÀ. LATENZE E DURATA SONO ESPRESSE IN SECONDI.

W	p50	p95	p99	Durata
1	0.57	1.21	1.40	125.82
2	0.39	1.08	1.60	125.53
3	0.39	0.83	1.16	125.54
4	0.34	0.83	1.10	125.45
6	0.35	1.01	1.58	126.18

Il miglioramento è evidente fino a quattro Worker: rispetto a W_1 , W_4 riduce la mediana di circa il 40% e il p95 di circa il 32%. Con W_6 viene raggiunto il massimo parallelismo consentito dalle sei partizioni Kafka, ma non si osserva un ulteriore miglioramento: la mediana rimane sostanzialmente invariata, mentre p95 e p99 risultano leggermente superiori rispetto a W_4 .

Il mancato miglioramento osservato con W_6 può essere ricondotto anche alla granularità del partizionamento: ciascuna

partizione Kafka è assegnata a un solo Worker del consumer group, quindi un eventuale sbilanciamento del carico tra le sei partizioni non può essere ulteriormente suddiviso aumentando il numero di Worker.

C. Impatto delle condizioni di rete

Le condizioni di rete sono emulate tramite Linux `tc-netem` nel network namespace dei container interessati. Le regole sono selettive e non alterano il traffico di orchestrazione o di raccolta delle metriche. Viene degradato soltanto il traffico di `edge-8` verso Kafka, mentre gli altri Edge e il collegamento Cloud-Global rimangono invariati. In questo modo è possibile osservare l'effetto di una singola sorgente rallentata sul progresso event-time della relativa partizione.

La campagna utilizza sei Cloud Worker e considera due profili di rete. Nel profilo *delay* vengono aggiunti 50 ms al traffico Edge-Kafka, mentre nel profilo *bandwidth* il traffico di `edge-8` verso Kafka viene limitato a 140 kbit/s. Per le prove con banda limitata la capacità della coda Edge viene dimensionata in modo da assorbire l'accumulo transitorio senza introdurre drop.

Per ciascun profilo vengono confrontate due politiche di avanzamento event-time. La configurazione *conservative* utilizza uno skew sufficientemente ampio da non forzare l'avanzamento della frontiera e attende quindi il progresso della sorgente rallentata. La configurazione *bounded* ammette invece uno skew massimo di 150 minuti in event time, equivalente a 450 ms reali con fattore di accelerazione 20 000. Quando tale limite viene superato, la frontiera può avanzare e i contributi che raggiungono successivamente una finestra già finalizzata vengono classificati come late e scartati.

Tutte le esecuzioni considerate terminano senza drop nei Simulator, negli Edge o nell'emulazione di rete e con lag finale nullo per entrambi i consumer group. Nelle configurazioni *bounded* i contributi late sono quindi attribuibili alla politica di avanzamento studiata e non a perdite infrastrutturali.

Tabella III
CONDIZIONI DI RETE EDGE-CLOUD: COMPLETEZZA E LATENZA GLOBALE. LATENZE E DURATA SONO ESPRESSE IN SECONDI.

Profilo	Politica	Compl.	Eventi late	p50	p95	p99	Durata
Delay 50 ms	Conservative	100.00%	0	1.71	3.87	4.23	126.31
Delay 50 ms	Bounded	89.12%	229 301	0.57	0.83	1.16	126.29
Banda 140 kbit/s	Conservative	100.00%	0	27.01	37.82	38.61	162.27
Banda 140 kbit/s	Bounded	82.44%	370 046	0.61	1.58	2.77	165.47

Con il ritardo aggiunto di 50 ms, la politica *conservative* mantiene il 100% degli eventi con un p95 pari a 3.87 s. La politica *bounded* riduce il p95 a 0.83 s, con una completezza dell'89.12%.

Con il collegamento limitato a 140 kbit/s, la configurazione *conservative* mantiene nuovamente il 100% degli eventi, ma il p95 raggiunge 37.82 s. La configurazione *bounded* riduce invece il p95 a 1.58 s, con una completezza dell'82.44%.

Le due configurazioni *bounded* registrano rispettivamente 229 301 e 370 000 eventi late in media. La riduzione della

latenza di emissione è quindi ottenuta consentendo alla frontiera event-time di avanzare senza attendere indefinitamente la sorgente rallentata, al costo di una riduzione misurabile della completezza dei risultati.

D. Discussione dei risultati

La campagna di scalabilità mostra che l'aumento del numero di Cloud Worker riduce la latenza di emissione senza modificare workload, partizionamento o risultati prodotti. Il beneficio è evidente fino a quattro Worker e presenta rendimenti decrescenti prima del limite di sei Worker imposto dal numero di partizioni. Il workload mantiene inoltre una distribuzione non perfettamente uniforme degli eventi tra le partizioni.

La campagna di rete mostra invece il compromesso introdotto dalla politica bounded. In presenza di una sorgente rallentata, la configurazione conservative mantiene la completezza attendendone il progresso, mentre la configurazione bounded può ridurre la latenza di emissione al costo di contributi successivamente classificati come late.

VI. LIMITI DEL DEPLOYMENT SPERIMENTALE

La tolleranza ai guasti non rientra tra i requisiti individuali selezionati per il progetto e non è stata oggetto della campagna sperimentale. Lo stato delle finestre in elaborazione è mantenuto in memoria dai Cloud Worker e dal Global Aggregator e non è previsto un meccanismo di checkpoint o migrazione dello stato durante una run. Un guasto di uno di questi componenti richiede quindi il riavvio dell'esecuzione anziché il recupero trasparente dello stato precedente.

La valutazione è inoltre condizionata dalle risorse disponibili nell'ambiente AWS utilizzato. I Cloud Worker vengono eseguiti sullo stesso host fisico virtuale e condividono quindi CPU, memoria e capacità di rete; l'aumento del numero di Worker misura pertanto la scalabilità del livello applicativo all'interno del deployment considerato e non equivale a una valutazione su un numero crescente di nodi EC2 indipendenti.

Il Cloud Core utilizza una configurazione infrastrutturale differente dagli altri host poiché concentra sia il broker Kafka sia il Global Aggregator. Nel deployment sperimentale è stata quindi utilizzata un'istanza `c6i.large`, con l'obiettivo di fornire capacità stabile al broker e ridurre il rischio che l'infrastruttura Kafka costituisca il collo di bottiglia durante la valutazione della scalabilità dei Cloud Worker. La scelta consente di concentrare l'analisi sul livello Worker, ma i risultati rimangono specifici della configurazione infrastrutturale utilizzata.

VII. CONCLUSIONI

Il progetto ha realizzato una pipeline distribuita per il monitoraggio ambientale nel Compute Continuum, suddividendo l'elaborazione tra Simulator, Edge e Cloud e utilizzando MQTT e Kafka per la comunicazione tra i diversi livelli.

La valutazione sperimentale ha mostrato che l'aumento del numero di Cloud Worker riduce la latenza di emissione delle finestre globali fino a quando il parallelismo disponibile risulta

effettivamente utile per il workload considerato. Gli esperimenti di rete hanno inoltre evidenziato il compromesso tra latenza e completezza introdotto dalla politica bounded: una sorgente rallentata può essere attesa dalla politica conservative oppure superata dalla frontiera event-time, riducendo la latenza al costo di contributi late.

Nel complesso, i risultati mostrano come la distribuzione dell'elaborazione tra Edge e Cloud, insieme alla gestione esplicita del progresso in event time, permetta di studiare sia la scalabilità del livello Cloud sia il comportamento del sistema in presenza di sorgenti con velocità differenti.

RIFERIMENTI BIBLIOGRAFICI

- [1] Sensor.Community, *Get historical datas*, documentazione della comunità sull'archivio delle osservazioni. <https://forum.sensor.community/t/get-historical-datas/1648>. Consultato l'11 settembre 2026.
- [2] Apache Software Foundation, *Apache Kafka – Design*, documentazione 4.3. <https://kafka.apache.org/43/design/design/>. Consultato l'11 settembre 2026.
- [3] Apache Software Foundation, *Apache Avro 1.12.0 – Specification*. <https://avro.apache.org/docs/1.12.0/specification/>. Consultato l'11 settembre 2026.